

AI Operations (AIOps)

Module 1 Student Handouts

Quick-reference sheets for Lectures 1, 2 & 3 — keep these next to you during labs

Handout 1 — MLOps Foundations & Technical Debt

Big idea: In production ML, most of the engineering effort is NOT model training — it's the surrounding system that keeps the model correct, safe, and reproducible.

The 6 Sources of Hidden Technical Debt

- Boundary Erosion — statistical logic and rule-based logic mix, blurring clean interfaces.
- Entanglement (CACE) — “Changing Anything Changes Everything.” No safe small changes.
- Hidden Feedback Loops — a model's predictions influence the data it's later trained on.
- Undeclared Consumers — other systems silently depend on a model's output.
- Data Dependencies — unstable upstream data is riskier than unstable code.
- Configuration Debt & Glue Code/Pipeline Jungles — scattered configs and stitched-together scripts with no orchestration layer.

MLOps vs DevOps — At a Glance

	DevOps	MLOps
Versions	Code	Code + Data + Model + Env
Fails by	Crashing / erroring	Quietly getting worse (drift)
Rollback unit	A deployment	A whole bundle

Model Decay — Two Types

- Data Drift — input feature distributions change over time.
- Concept Drift — the relationship between inputs and the correct label changes.

Reproducibility Protocol Preview (built across this module)

1. Pin the code — Git commit.
2. Pin the environment — Conda/Mamba environment.yml.
3. Pin the data — DVC (Lecture 3).
4. Pin the run — MLflow (Lecture 2).

Lab 1 Command Cheat Sheet

```
git init
git add <files>
git commit -m "message"
mamba create -n aiops-m1 python=3.10 -y
```

```
mamba activate aiops-m1  
mamba env export --no-builds > environment.yml  
# On a new machine, to reproduce:  
mamba env create -f environment.yml -n aiops-m1-verify
```

Handout 2 — Experiment Tracking with MLflow

Big idea: If a run isn't logged, it didn't happen — you can't compare it, reproduce it, or trust it later.

MLflow Vocabulary

Term	Meaning
Experiment	A named group of related runs
Run	One execution of training/eval code
Parameter	Static config for a run (lr, batch size)
Metric	A value tracked over time/steps (loss, accuracy)
Artifact	A file output (model, plot, sample predictions)
Tag	Free-form metadata (git commit, team, seed)

MLflow Architecture (4 parts)

- Tracking Server — receives/serves logged data via UI + REST API.
- Backend Store — stores params/metrics/tags metadata.
- Artifact Store — stores large binary files (models, plots).
- Model Registry — versioned, stage-aware model catalogue (Lecture 3).

Lab 2 Command / Code Cheat Sheet

```
mlflow server --backend-store-uri sqlite:///mlflow.db --default-artifact-root ./mlruns --host 0.0.0.0 --port 5000
```

```
import mlflow

mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("my-experiment")
with mlflow.start_run(run_name="run-1"):
    mlflow.log_param("lr", 0.001)
    mlflow.log_metric("val_accuracy", 0.91, step=epoch)
    mlflow.log_artifact("confusion_matrix.png")
    mlflow.pytorch.log_model(model, "model")
```

Checklist Before You Leave Lab 2

- At least 4 runs logged, varying one or more hyperparameters.
- Every run has params, metrics, and at least one artifact.
- You have screenshotted the run comparison view (needed for Assignment 1, Q2).
- A git_commit tag and a seed param are logged on at least one run.

Handout 3 — Reproducibility: Data & Model Versioning

Big idea: Code, Data, and Model must be versioned TOGETHER — a Git commit alone is not enough to reproduce a result.

The Version Control Triad

What	Tool	Unit versioned
Code	Git	Commit
Data	DVC	.dvc pointer file (hash) + remote storage
Model	MLflow Model Registry	Registered model version + stage

DVC Core Workflow

```
dvc init
dvc remote add -d storage /path/to/remote
dvc add data/train.csv
git add data/train.csv.dvc .gitignore
git commit -m "v1 dataset"
dvc push
# To reproduce an OLD version exactly:
git checkout <old-commit-hash>
dvc checkout
```

MLflow Model Registry — Stages

Stage	Meaning
None	Registered, not yet reviewed
Staging	Under validation
Production	Currently serving traffic
Archived	Retired, kept for audit/rollback

```
result = mlflow.register_model(model_uri="runs:/<run_id>/model", name="my-model")
client.transition_model_version_stage(name="my-model", version=result.version, stage="Staging")
```

Audit Trail Cheat Sheet (for the healthcare case study)

Auditor's Question	Where to Find the Answer
Which code produced this model?	Git commit hash (tagged in the MLflow run)
Which exact data was used?	.dvc file committed with that code + dvc checkout
What hyperparameters/metrics?	MLflow run page
Was this model ever in Production?	MLflow Model Registry stage-transition history

End-to-End Capstone Drill Checklist

1. Train + log a run in MLflow (params, metrics, artifact, seed, git_commit tag).

2. dvc add the dataset; commit the .dvc file in the SAME Git commit as the training code.
3. Register the model; transition it to Staging.
4. Share your repo with a partner.
5. Partner reproduces using only: git clone → git checkout → dvc checkout → mamba env create → rerun script.
6. Partner logs a confirmation note (matched or discrepancy) in the MLflow run.